



CS4135 - Software Architectures Project

Team members

Caylum Hurley - 22356363

Vivian Jently - 22079017

Oscar Zhou -22338705

Joe Considine - 22344977

Darragh Quinn - 22359621

Daragh Downes - 22351159

Department of Computer Science and Information Systems Faculty of Science and
Engineering
University of Limerick

1. Ownership & Division of Bounded Contexts	3
1.1 Bounded Context Map	4
1.2 Cross-Context Dependencies	4
2. DDD Modelling within Each Context	5
2.1 Identity & Access Context	5
2.2 Catalogue Context	7
2.3 Lending Context.....	8
2.4 Notification Context	9
2.5 Book Club Context	10
3. Aggregate Design & Invariants	11
3.1 Aggregate Boundary Justifications	12
3.2 Invariant Summary	12
4. Inter-Context Contracts	14
4.1 REST API Contracts (Synchronous)	15
4.2 Domain Event Contracts (Asynchronous)	15
4.3 Anti-Corruption Layer Design	16
5. Conformance to UML/DDD Notation	17
5.1 Ubiquitous Language Glossary	18
5.2 UML models	19
6. Team Contribution	20

1. Ownership & Division of Bounded Contexts

To ensure that each domain has clear ownership and responsibility, the decision was made to decompose our E-Library system into five bounded contexts,

1.1 Bounded Context Map

The five bounded contexts and their ownership:

Bounded Context	Owner	Responsibility	Primary Entities
Identity & Access	Joe Considine	Authentication, authorisation, user profiles, role management	User, Role, JWT Token
Catalogue	Oscar Zhou	Book inventory, search, availability, CRUD operations	Book, Author, Category, Copy
Lending	Daragh Downes	Borrow/return workflows, due dates, loan lifecycle	Loan, LoanStatus, BorrowRequest
Notification	Vivian Jently	Due-date reminders, overdue alerts, confirmations	Notification, NotificationEvent
Book Club	Darragh Quinn, Caylum Hurley	Club creation, memberships, chat/messaging within clubs	BookClub, ClubMembership, Message
API Gateway	Caylum Hurley	Central entry point for all client requests; handles authentication, request routing, and security enforcement before forwarding to bounded contexts	JWT filter, Routing layer, Security context, Gateway request

1.2 Cross-Context Dependencies

The table below shows every cross-context dependency, type of relationship, and the anti-corruption strategy that was used.

Upstream Context	Downstream Context	Relationship Type	Dependency Description
Identity & Access	Catalogue	Customer/Supplier	Catalogue validates JWT tokens via the Auth Service before permitting admin mutations
Identity & Access	Lending	Customer/Supplier	Lending resolves user identity and role before issuing a loan
Identity & Access	Book Club	Customer/Supplier	Book Club verifies membership privileges through the Identity context
Catalogue	Lending	Published Language	Lending references Book Copy identifiers via a shared DTO contract
Lending	Notification	Event-Driven	Lending emits domain events (LoanCreated, LoanOverdue) that Notification subscribes to
Book Club	Notification	Event-Driven	Club events (MemberJoined, MessagePosted) trigger Notification events

2. DDD Modelling within Each Context

Each bounded context is made up of all the DDD parts: Entities, Value Objects, Aggregates (and their roots), Repositories, Domain Services, and Domain Events.

2.1 Identity & Access Context

Aggregate: UserAccount

In this case, the UserAccount aggregate is the only one. It contains rules for user identity and access.

DDD Construct	Name	Description
Aggregate Root (Entity)	UserAccount	The main entity that owns the lifecycle of identity data. UserId (UUID), email (Email VO), passwordHash (String), fullName (String), role (Role VO), and createdAt (DateTime) are the fields.
Value Object	Email	Email address that can't be changed. Makes sure the format stays the same. No identity.
Value Object	Role	Enum value: STUDENT STAFF ADMIN. Once set by admin, it is immutable.
Value Object	HashedPassword	Encapsulates bcrypt hash. Exposes verify(plaintext): Boolean. Never exposes raw hash.
Domain Service	AuthenticationService	Verifies credentials, issues JWT. Operates across UserAccount and JWT infrastructure. Stateless.
Domain Service	TokenValidationService	Validates incoming JWT tokens; extracts Role and userId claim for downstream contexts.
Repository	UserAccountRepository	Persistence abstraction: findByEmail, findById, save, delete.
Domain Event	UserRegistered	When a new UserAccount is saved, this is fired. Payload: email, userId, and role.
Domain Event	PasswordChanged	Fired on successful password reset. Payload: userId, timestamp.

Invariants (Identity & Access)

- Every UserAccount instance must have a different email address.
- AuthenticationService makes sure that only an existing ADMIN can give someone a STAFF or ADMIN role.
- Never let anyone see HashedPassword outside of the UserAccount aggregate boundary.
- JWT expiration is a system-level non-functional constraint. The TokenValidationService makes sure that expired tokens don't get to domain logic.

2.2 Catalogue Context

Aggregate: BookCopy

BookCopy is the transactional aggregate, which means it's one copy, either physical or digital. Book is a separate entity that gives descriptive information and is owned by Catalogue.

DDD Construct	Name	Description
Aggregate Root (Entity)	BookCopy	Represents a borrowable copy. Fields: copyId (UUID), bookId (ref), availability (AvailabilityStatus VO), condition (String).
Entity	Book	Descriptive aggregate for a title. Fields: bookId (UUID), isbn (ISBN VO), title, description, publicationYear, totalCopies, availableCopies.
Entity	Author	authorId (UUID), name. Associated to Book (0..*).
Entity	Category	categoryId (UUID), name. Tag-style; a Book may have 0..* Categories.
Value Object	ISBN	Immutable 10 or 13-digit validated identifier. Format enforced at construction.
Value Object	AvailabilityStatus	Enum: AVAILABLE BORROWED RESERVED. Domain rules protect transitions
Domain Service	BookSearchService	Search books by title, author, category, and availability. Returns read-model projections.

DDD Construct	Name	Description
Domain Service	InventoryService	Adjusts availableCopies on Book when a BookCopy changes AvailabilityStatus.
Repository	BookRepository	findByld, findByIsbn, search(criteria), save, delete.
Repository	BookCopyRepository	findAvailableCopiesByBookId, findByld, save.
Domain Event	BookAdded	Fired when a new Book is created by an Admin. Payload: bookId, isbn, title.
Domain Event	CopyAvailabilityChanged	Fired when the status of a BookCopy changes. Payload: copyId, bookId, and newStatus. Lending took over.

Invariants (Catalogue)

- availableCopies on Book must equal the count of BookCopy instances with AvailabilityStatus = AVAILABLE.
- Every Book entity must have a different ISBN.
- A BookCopy can't go straight from BORROWED to RESERVED; it has to go through AVAILABLE first.
- At all times, totalCopies must be >= availableCopies.

2.3 Lending Context

Aggregate: Loan

The loan is the central part of the borrow-and-return process. It only uses the IDs of BookCopy and UserAccount to refer to them (no cross-context object references).

DDD Construct	Name	Description
Aggregate Root (Entity)	Loan	Fields: loanId (UUID), userId (ref), copyId (ref), issueDate (Date), dueDate (DueDate VO), returnDate (Date [0..1]), status (LoanStatus VO).
Value Object	DueDate	Immutable. Calculated at issuance (issueDate + 14 days default). Exposes isOverdue(): Boolean.

DDD Construct	Name	Description
Value Object	LoanStatus	Enum: ACTIVE RETURNED OVERDUE. Transitions enforced by Loan aggregate.
Domain Service	LoanIssuanceService	Checks to see if a user is eligible and if a copy is available (through an ACL/API call to the Catalogue). Makes a Loan and fires LoanCreated.
Domain Service	ReturnProcessingService	Processes return: sets returnDate, changes LoanStatus to RETURNED, publishes CopyReleased event.
Domain Service	OverdueDetectionService	Scheduled service: checks active Loans where DueDate.isOverdue() = true, changes status to OVERDUE, and sends LoanOverdue.
Repository	LoanRepository	findById, findActiveByUserId, findOverdue, save.
Domain Event	LoanCreated	Payload: loanId, userId, copyId, dueDate. Consumed by Notification.
Domain Event	LoanOverdue	Payload: loanId, userId, copyId, dueDate. Consumed by Notification.
Domain Event	CopyReleased	Payload: copyId. Consumed by Catalogue to restore AVAILABLE status.

Invariants (Lending)

- LoanIssuanceService ensures that a user can't have more than five active loans at the same time.
- A Loan cannot be issued for a BookCopy with AvailabilityStatus != AVAILABLE.
- returnDate must be >= issueDate.
- Once LoanStatus = RETURNED, the Loan is immutable.
- The system actor OverdueDetectionService sets LoanStatus = OVERDUE, never a user action.

2.4 Notification Context

Aggregate: Notification

Notification is a lightweight context. Its aggregate is a record of a notification that was sent out, made up of domain events from Lending and Book Club that came in.

DDD Construct	Name	Description
Aggregate Root (Entity)	Notification	Fields: notificationId (UUID), userId (ref), type (NotificationType VO), message (String), createdAt (DateTime), sentAt (DateTime [0..1]), status (NotificationStatus VO).
Value Object	NotificationType	Enum: DUE_SOON_REMINDER OVERDUE_ALERT BORROW_CONFIRMATION RETURN_CONFIRMATION CLUB_ANNOUNCEMENT.
Value Object	NotificationStatus	Enum: PENDING SENT FAILED. Transitions guarded by dispatch logic.
Domain Service	NotificationDispatchService	Creates Notification entities from incoming events. Ways to get to the delivery channel (email or in-app). Marks SENT or FAILED.
Repository	NotificationRepository	findById, findByUserId, findPending, save.
Domain Event (consumed)	LoanCreated (from Lending)	Triggers BORROW_CONFIRMATION notification.
Domain Event (consumed)	LoanOverdue (from Lending)	Triggers OVERDUE_ALERT notification 3 days before and on due date.

Invariants (Notification)

- You have 30 seconds to send a BORROW_CONFIRMATION notification after you get a LoanCreated event.
- A DUE_SOON_REMINDER can only be sent once per loan per trigger window (3-day mark).
- Notifications that fail are retried again up to three times before they are marked as permanently failed.

2.5 Book Club Context

Aggregates: BookClub, ClubMembership, Message

DDD Construct	Name	Description
Aggregate Root (Entity)	BookClub	clubId (UUID), name (String), description (String [0..1]), createdAt (DateTime). Owns ClubMembership collection.
Entity	ClubMembership	membershipId (UUID), userId (ref), clubId (ref), role (ClubRole VO), joinedAt (DateTime). Child of BookClub aggregate.
Aggregate Root (Entity)	Message	messageId (UUID), content (MessageContent VO), clubId (ref), authorUserId (ref), sentAt (DateTime), editedAt (DateTime [0..1]).
Value Object	ClubRole	Enum: OWNER MODERATOR MEMBER.
Value Object	MessageContent	Immutable string; enforces max 2000 character limit and non-empty invariant.
Domain Service	ClubModerationService	Handles member role changes, member removal. Only OWNER or MODERATOR may invoke.
Domain Service	MessageService	Handles post/edit/delete of Messages. Edit sets editedAt; delete is soft-delete. Only author or MODERATOR may delete.
Repository	BookClubRepository	findById, findAll, save, delete.
Repository	MessageRepository	findByClubId(paginated), findById, save.
Domain Event	MemberJoined	Payload: clubId, userId. Consumed by Notification for welcome message.
Domain Event	MessagePosted	Payload: messageId, clubId, authorUserId. May trigger club-level notification.

Invariants (Book Club)

- ☐ Each BookClub must have exactly one OWNER (ClubRole = OWNER) at all times.
- ☐ An OWNER cannot leave a club unless they first transfer ownership to another MEMBER.
- ☐ MessageContent must be non-empty and no longer than 2000 characters.
- ☐ A user may join a given BookClub at most once (unique userId per clubId in ClubMembership).

3. Aggregate Design & Invariants

This section explains why the aggregate boundary decisions were made and shows how the system's transactional consistency rules work.

3.1 Aggregate Boundary Justifications

Aggregate Root	Boundary Justification
UserAccount	At the same time, all identity fields, such as the password, email, and role, are changed. These fields don't need to be changed in any other context.
BookCopy	BookCopy is the unit that tracks availability. A book can have many copies, and the status of each copy can change on its own without locking the whole book.
Book	The Book group is separate because the descriptive data (title, ISBN, author list) changes on its own, no matter if a certain copy is available or not.
Loan	A Loan shows you how to borrow money from start to finish. There is only one transaction for all changes in state (ACTIVE -> OVERDUE -> RETURNED) on this aggregate.
Notification	Every Notification is sent out by itself. You don't have to lock a group of notifications in a transaction.
BookClub	A BookClub and its ClubMembership collection are modified together, adding/removing members is an operation on the club. This justifies a single aggregate.
Message	Messages are independent from the BookClub aggregate (a club can have thousands of messages). Separating them prevents contention on the BookClub aggregate root.

3.2 Invariant Summary

Context	Invariant	Enforcement Point
Identity & Access	Email uniqueness across all UserAccount instances	UserAccountRepository + domain precondition check

Context	Invariant	Enforcement Point
Identity & Access	Role elevation requires ADMIN actor	AuthenticationService
Catalogue	availableCopies == count(BookCopy[AVAILABLE])	InventoryService on CopyAvailabilityChanged event
Catalogue	ISBN uniqueness	BookRepository precondition
Catalogue	BORROWED - > AVAILABLE only (no direct RESERVED)	AvailabilityStatus VO transition guard
Lending	Max 5 active loans per user	LoanIssuanceService precondition
Lending	Loan requires AVAILABLE copy	LoanIssuanceService via Catalogue ACL call
Lending	RETURNED loans are immutable	Loan entity state guard
Notification	DUE_SOON_REMINDER sent at most once per loan per window	NotificationRepository deduplication query
Notification	Max 3 retries on FAILED	NotificationDispatchService retry counter
Book Club	Exactly 1 OWNER per BookClub at all times	ClubModerationService ownership transfer guard
Book Club	User joins club at most once	ClubMembershipRepository unique constraint + domain check
Book Club	MessageContent: 1–2000 chars	MessageContent Value Object constructor

4. Inter-Context Contracts

A written contract governs every interaction between different contexts. This section discusses how the API/event contracts, the Anti-Corruption Layer strategies, and what each side of the contract must do.

4.1 REST API Contracts (Synchronous)

Contract	Upstream	Downstream	Endpoint	Response DTO	ACL / Strategy
Token Validation	Identity & Access	All Contexts (via Gateway)	GET /auth/validate	{ userId, role, valid: bool }	Gateway intercepts; downstream receives validated claims only, never raw JWT internals
Copy Availability Check	Catalogue	Lending	GET /catalogue/copies/{copyId}/availability	{ copyId, status, bookTitle }	Lending ACL maps CatalogueAvailabilityDTO - internal CopyStatusVO, avoids Catalogue model leaking into Lending
User Profile Read	Identity & Access	Book Club	GET /users/{userId}/profile	{ userId, fullName, role }	Book Club ACL maps to internal ClubMemberProfileVO, only the fields needed are extracted

4.2 Domain Event Contracts (Asynchronous)

Event	Publisher	Subscriber	Payload Schema	Handling Strategy
LoanCreated	Lending	Notification	{ loanId, userId, copyId, bookTitle,	Notification builds BORROW_CONFIRMATION.

Event	Publisher	Subscriber	Payload Schema	Handling Strategy
			dueDate, timestamp }	Idempotent: loanId used as deduplication key.
LoanOverdue	Lending	Notification	{ loanId, userId, copyId, bookTitle, dueDate, daysOverdue, timestamp }	Notification builds OVERDUE_ALERT. Deduplication by loanId + notification type.
CopyReleased	Lending	Catalogue	{ copyId, loanId, returnedAt }	Catalogue ACL translates to AvailabilityStatus = AVAILABLE transition on BookCopy.
MemberJoined	Book Club	Notification	{ clubId, clubName, userId, joinedAt }	Notification builds CLUB_ANNOUNCEMENT for the joining user.
UserRegistered	Identity & Access	Notification	{ userId, email, fullName, timestamp }	Notification sends welcome message (optional, BORROW_CONFIRMATION type).

4.3 Anti-Corruption Layer Design

An ACL is used by each downstream context to keep its domain model safe from changes made upstream. Before any domain logic is called, the ACL sits at the edge of the downstream context and does the translation.

Downstream Context	ACL Class	Purpose
Lending	CatalogueACL	Translates Catalogue REST DTO -> Lending's internal CopyReference value object. If Catalogue changes its DTO, only CatalogueACL is updated.

Downstream Context	ACL Class	Purpose
Catalogue	LendingEventTranslator	Translates CopyReleased event payload -> AvailabilityUpdateCommand for the Catalogue domain service.
Notification	LendingEventTranslator	Translates LoanCreated/LoanOverdue -> NotificationRequest internal model.
Notification	ClubEventTranslator	Translates MemberJoined/MessagePosted - NotificationRequest internal model.
Book Club	IdentityACL	Translates User profile REST response - ClubMemberProfileVO used only within the Book Club context.

5. Conformance to UML/DDD Notation

All UML models made for this assignment use the same notation standard, which comes from DDD modelling conventions. This section talks about the notation guide that was used and checks the readability standards.

5.1 Ubiquitous Language Glossary

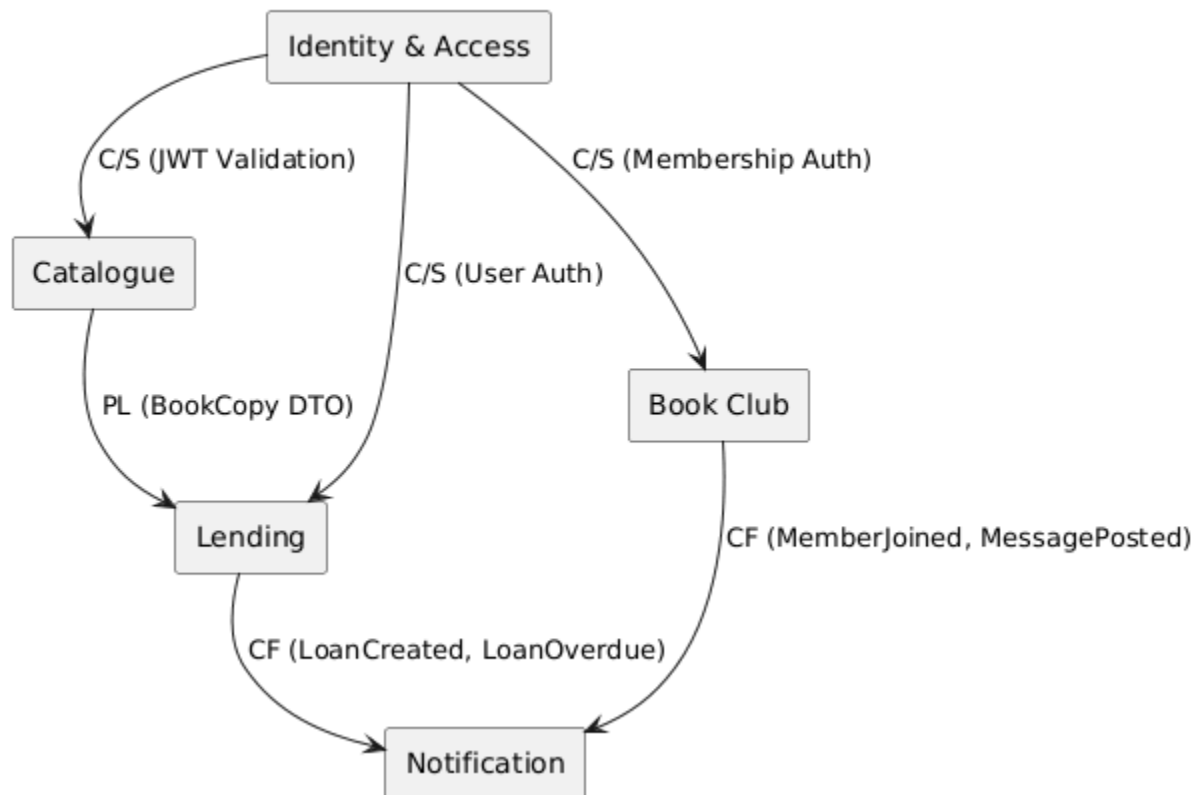
All UML models made for this assignment use the same notation standard, which comes from DDD modelling conventions. This section talks about the notation guide that was used and checks the readability standards.

Term	Context(s)	Definition
Loan	Lending	A record representing a user borrowing one BookCopy, from issueDate to returnDate.
BookCopy	Catalogue / Lending	A single borrowable instance of a Book title (physical or digital).
Book	Catalogue	The descriptive metadata for a title: ISBN, title, authors, categories.
UserAccount	Identity & Access	The domain entity representing a registered user with credentials and a Role.
Role	Identity & Access	An enum attribute of UserAccount: STUDENT, STAFF, or ADMIN.
DueDate	Lending	A Value Object representing the date by which a Loan must be returned.
ClubMembership	Book Club	The association between a UserAccount and a BookClub, with a ClubRole.
Notification	Notification	A dispatched or queued message sent to a user about a system event.
AvailabilityStatus	Catalogue	The current borrowability state of a BookCopy: AVAILABLE, BORROWED, or RESERVED.
ACL (Anti-Corruption Layer)	All	A translation layer at a context boundary preventing model pollution from upstream contexts.
Published Language	All	A versioned, shared DTO/event schema agreed between producer and consumer.

Term	Context(s)	Definition
Domain Event	All	An immutable record of something that occurred in the domain, named in past tense.

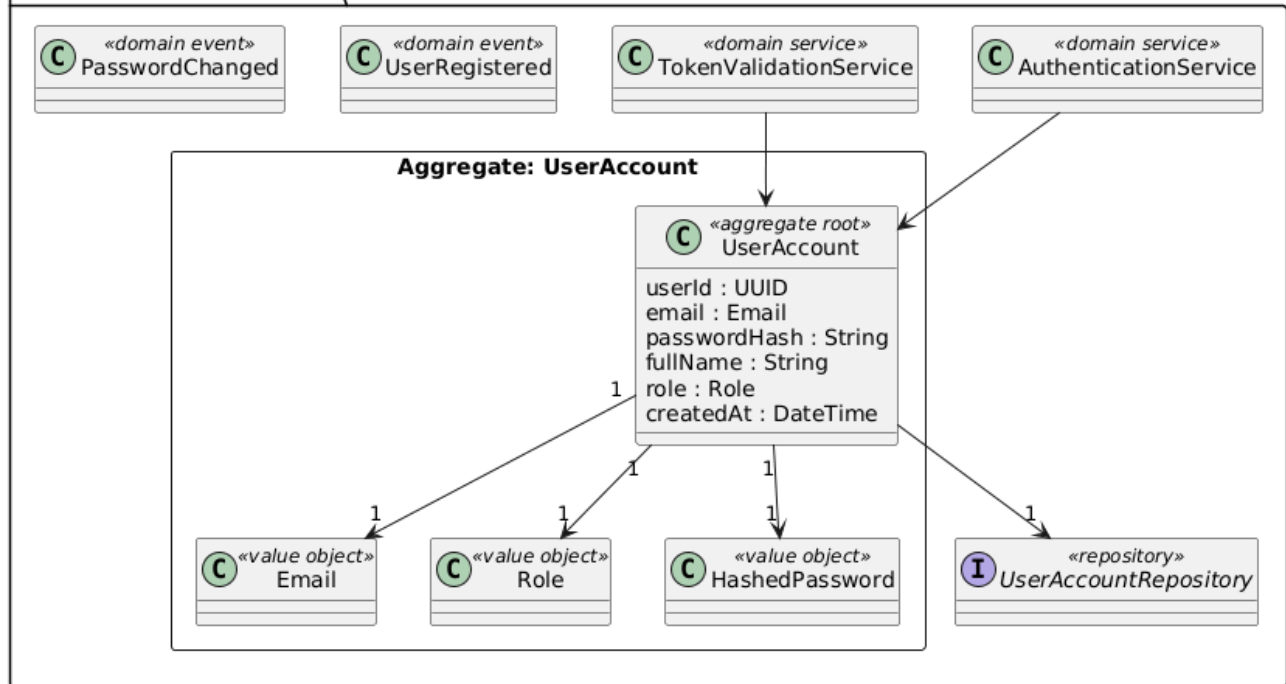
5.2 UML models

Context Map Diagram (v1.0)



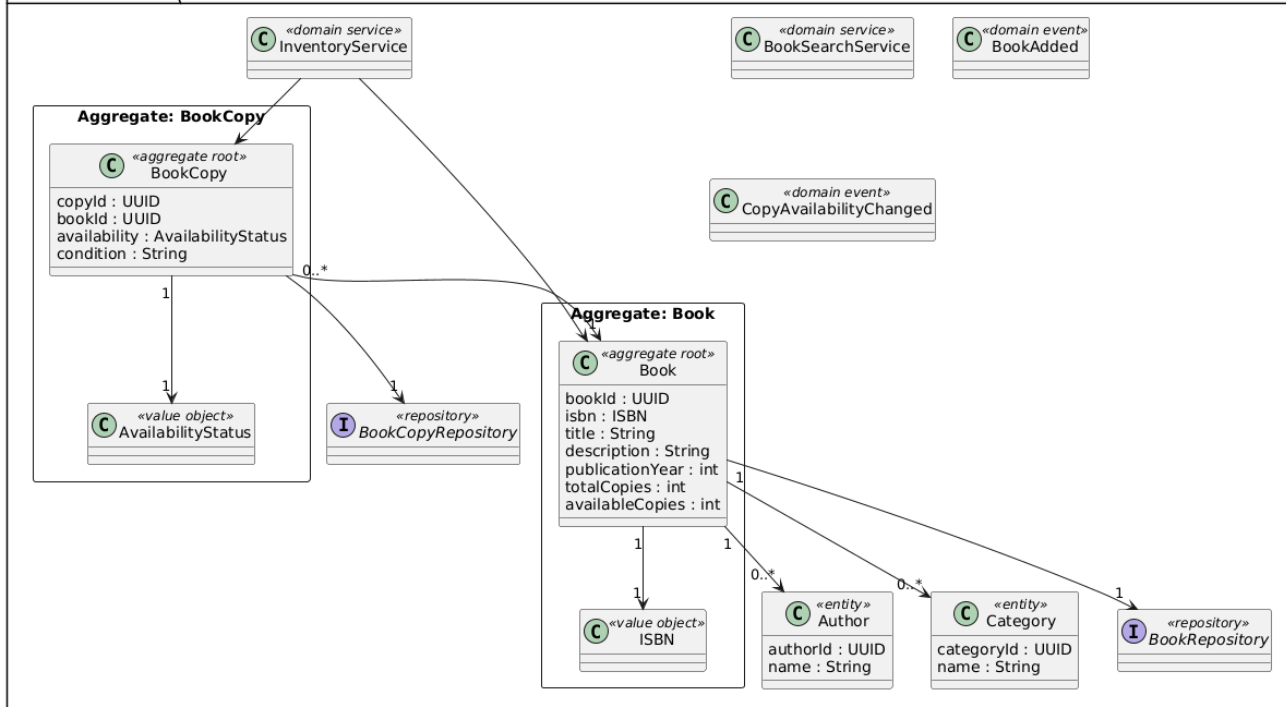
Identity & Access Context - Class Diagram (v1.0)

Identity & Access Context

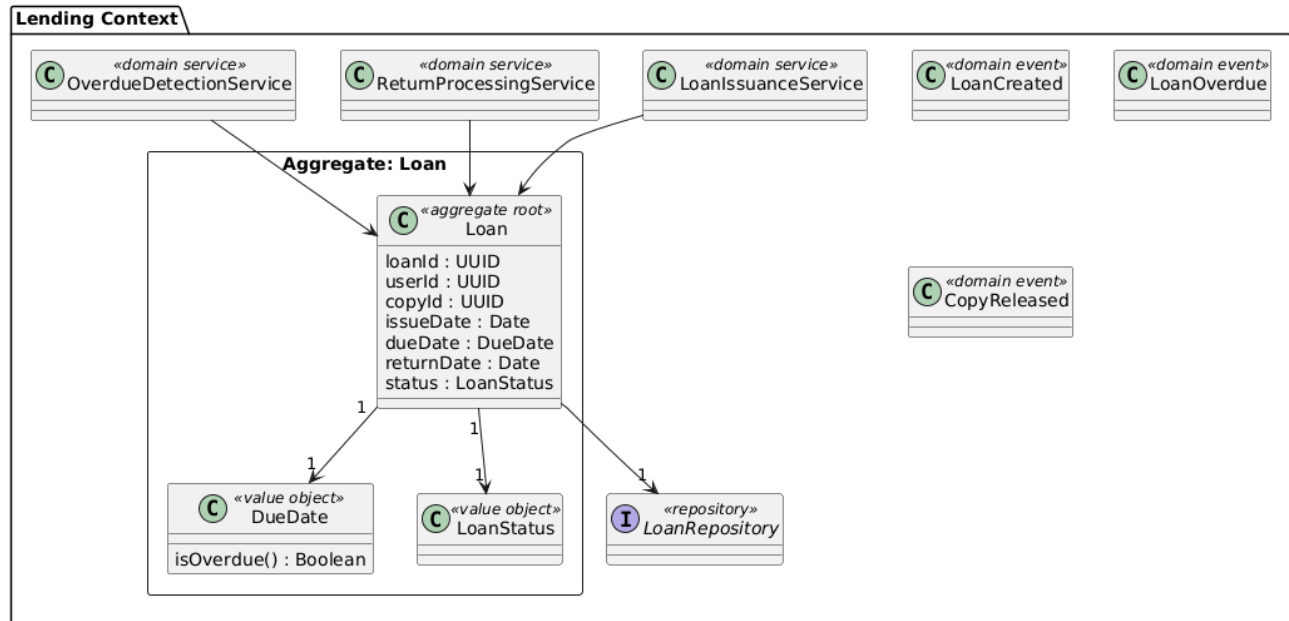


Catalogue Context - Class Diagram (v1.0)

Catalogue Context

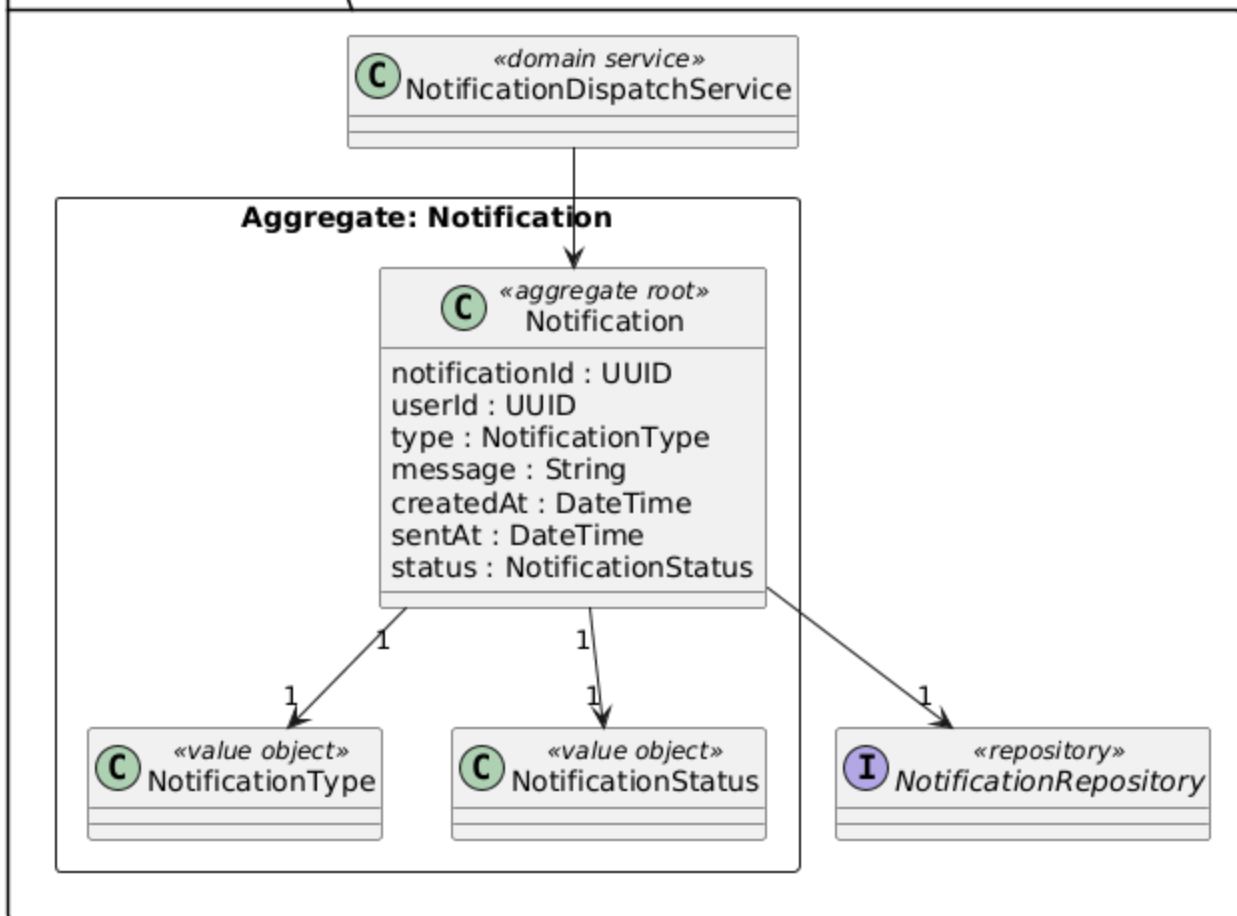


Lending Context - Class Diagram (v1.0)

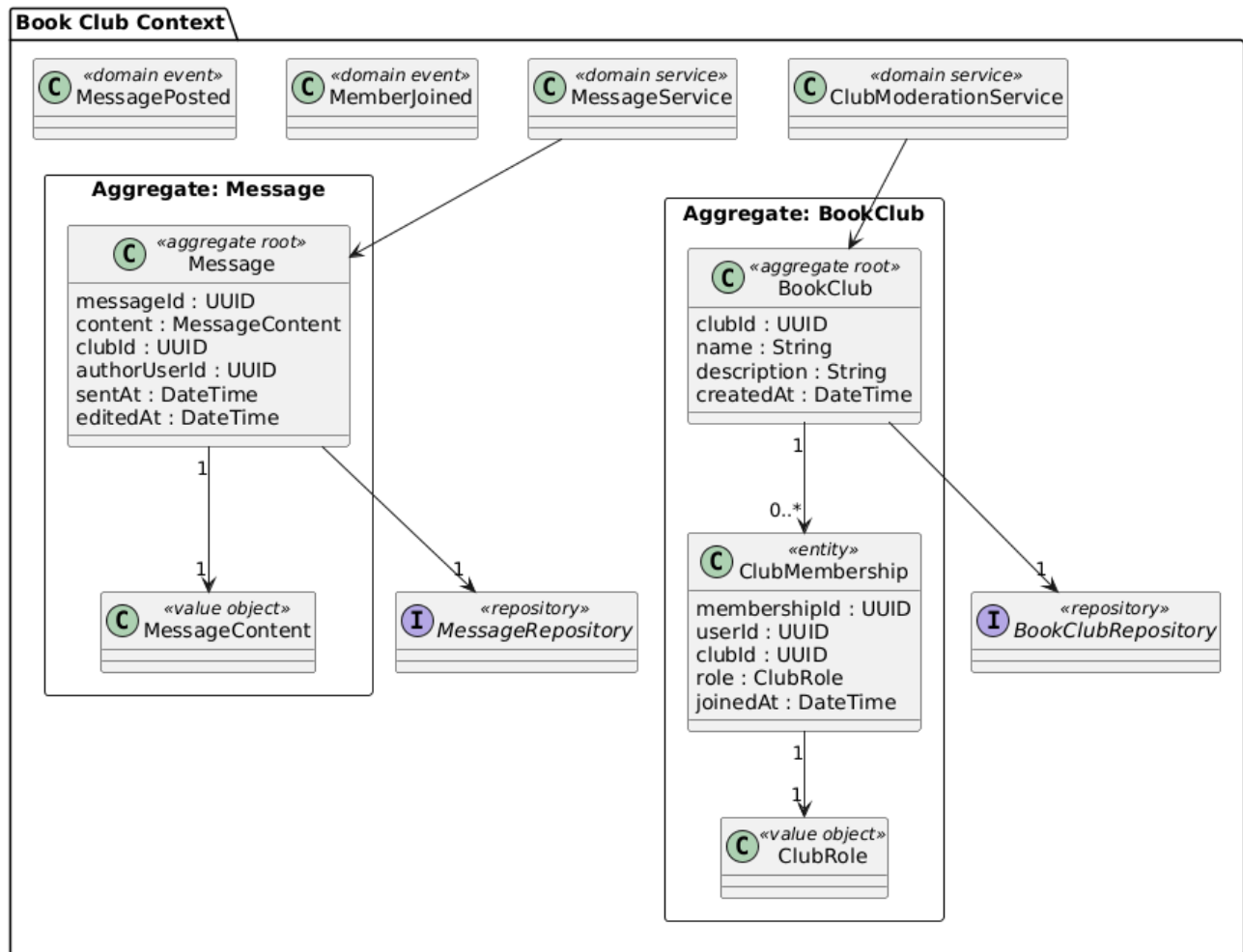


Notification Context - Class Diagram (v1.0)

Notification Context



Book Club Context - Class Diagram (v1.0)



6. API Gateway

The E-Library system has a central API Gateway that all client requests pass through. Instead of allowing external users see each bounded context directly, every incoming HTTP request goes through the gateway first. This method makes sure that security boundaries are always the same, separates clients from the internal service topology, and gives you one place to deal with things like authentication and routing that affect many parts of the system.

6.1 Gateway Responsibilities

The gateway does the following things before sending any request to a downstream bounded context:

Before sending any request to a downstream bounded context, the gateway does the following:

JWT Validation (JWT Filter): The gateway uses a dedicated JWT authentication filter (e.g. `OncePerRequestFilter` in Spring Security) to intercept every incoming HTTP request to protected routes. The filter extracts the Bearer token from the Authorisation header and validates it locally using a shared HMAC-SHA secret key. If the token is missing, invalid, or expired, the filter immediately terminates the request and returns a 401 Unauthorised response. Requests to public authentication paths (`/api/user/auth/**`) bypass this filter.

Role Verification: The gateway checks the token and gets the role claim. It then checks that it is one of the three system roles that are known (STUDENT, STAFF, ADMIN). If a role is missing or not recognised, a 401 Unauthorised error occurs.

The JWT filter operates within the gateway's security filter chain, ensuring that authentication is enforced consistently before any request is routed to downstream services.

Path prefix	Downstream service
<code>/api/user/</code>	elib-identity
<code>/api/book-catalogue/</code>	elib-catalogue
<code>/api/lending/</code>	elib-lending
<code>/api/notifications/</code>	elib-notification
<code>/api/bookclub/</code>	elib-bookclub

6.2 Request Lifecycle

The following sequence shows how a normal authenticated request moves through the system:

Receive: The client makes an HTTP request to the gateway endpoint.

Public path check: If the request path starts with /api/user/auth, it is sent on without checking for a token. This means that login and registration calls can go through without being verified.

Authenticate (JWT Filter Execution): The request processing pipeline starts the JWT authentication filter. It extracts the Bearer token from the Authorisation header and checks it against the shared HMAC secret. If the token is missing, broken, or expired, the filter stops the request and sends back a 401 Unauthorised response before it gets to any routing logic.

Check the role: The gateway gets the role claim from the parsed token. If the claim is missing or doesn't match STUDENT, STAFF, or ADMIN, a 401 Unauthorised message is sent back.

Route: The validated JWT claims are added to the request as a servlet attribute. The gateway gets the target service instance from Eureka and sends the request to the right bounded context. The raw Authorisation header is sent along as-is, and downstream services get the original request without any claim headers being added.

Respond: The response from the downstream context is sent back to the client without any changes.

6.3 Relationship to the Cross-Context Dependency Map

The gateway centralises JWT validation, which means that all incoming requests must be authenticated before they can reach any bounded context. The gateway checks the request locally using a shared secret instead of calling the Identity context at runtime. This means that the Identity service is not involved in every request and does not become a synchronous bottleneck. But downstream services that need to know who is calling must re-parse the Authorisation header themselves instead of using pre-extracted values from the gateway because validated claims are not sent as headers.

7. Team Contribution

Team Member	Contribution
Joe Considine	A2 - Identity & Access context full DDD model (UserAccount aggregate, value objects, domain services); A3 - Identity invariants and authentication domain logic
Daragh Downes	A1 - Bounded context ownership map; A2 - Lending context full DDD model (Loan aggregate, domain services); A3 - Lending invariants and loan lifecycle design
Caylum Hurley	A1 - Context map diagram and cross-context dependency table; A4 - API Gateway design and implementation; A4 Book Club event contracts; A5 - Ubiquitous language glossary and UML modelling
Darragh Quinn	A2 - Book Club context DDD model (BookClub, ClubMembership, Message aggregates); A4 Book Club event contracts
Vivian Jently	A2 - Notification context DDD model; A4 - event contract specifications (LoanCreated, LoanOverdue, MemberJoined)
Oscar Zhou	A2 - Catalogue context DDD model (Book, BookCopy, Author, Category); A3 - Aggregate boundary justifications; ISBN and AvailabilityStatus invariants